

03_model_training

November 24, 2025

0.1 1. Setup and Artefact Load.

Prepare the training environment and load the unified dataset artefacts created during 01_data_eda.ipynb. ##### Rationale: - Establish a **reproducible** experimental context (fixed random seed and deterministic cuDNN - NVIDIA CUDA Deep Neural Network library). - Resolve canonical project paths and verify existence of required directories (`data`, `experiments/pools/`, etc.). - Load the **pooled table** (`source_pool_rel.csv`) and the **preprocessing configuration** (`preprocessing_config.json`) that define the 3-class problem (Normal, Pneumonia, TB). - Create the output directories for checkpoints and metrics to keep training runs auditable. - Print a concise summary confirming *GPU availability*, *ImageNet normalisation*, and *reproducibility* settings.

```
[13]: from pathlib import Path
import os, json, random, numpy as np, pandas as pd, torch

# 1. Reproducibility.
# Use a fixed seed across all random libraries (Python, NumPy, and PyTorch)
# to make training results numerically repeatable on this workstation and
# future reruns.
def set_seed(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True # Disable non-deterministic
    kernels.
    torch.backends.cudnn.benchmark = False # Prevent cuDNN autotune drift.

SEED = 42
set_seed(SEED)

# 2. Resolve canonical project paths.
# This notebook lives in notebooks/, therefore "." points to the repository
# root.
PROJECT_ROOT = Path(".").resolve()
CHEXPRT_PATH = PROJECT_ROOT / "data" / "chexpert"
TBX11K_PATH = PROJECT_ROOT / "data" / "tbx11k"
POOLS_DIR = PROJECT_ROOT / "experiments" / "pools"
CHECKPOINTS_DIR = PROJECT_ROOT / "experiments" / "checkpoints"
```

```

RESULTS_DIR      = PROJECT_ROOT / "experiments" / "results"

# 3. Load persisted artefacts from 01_data_EDA.
# The pooled CSV contains relative paths, class labels, and source tags.
# The JSON holds preprocessing constants (IMG_SIZE, ImageNet mean/std, label_
    ↪map, transform mode).
pool_csv = POOLS_DIR / "source_pool_rel.csv"
cfg_path = POOLS_DIR / "preprocessing_config.json"

if not pool_csv.exists() or not cfg_path.exists():
    raise FileNotFoundError("Missing pooled artefacts. Please run 01_data_ed.
    ↪ipynb first "
                            "to generate experiments/pools/source_pool_rel.csv_
    ↪and preprocessing_config.json."
    )
source_pool_rel = pd.read_csv(pool_csv)
cfg = json.loads(cfg_path.read_text())

CLASS_TO_INDEX = {k: int(v) for k, v in cfg["class_to_index"].items()}
INDEX_TO_CLASS = {v: k for k, v in CLASS_TO_INDEX.items()}
IMG_SIZE       = int(cfg["img_size"])
IMAGENET_MEAN  = cfg["imagenet_mean"]
IMAGENET_STD   = cfg["imagenet_std"]
TRANSFORM_MODE = cfg.get("transform_mode", "center-crop")

# 5. Device and precision information.
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# Automatic Mixed Precision (AMP). Safely enable it for faster training.
USE_AMP = bool(device.type == "cuda") # Enable mixed precision if a GPU is_
    ↪present.

# 6. Compact readiness log.
print("03_model_training - Environment")
print(f'- Device: {device.type.upper()} | PyTorch: {torch.__version__} | CUDA:_
    ↪{torch.version.cuda}')
if device.type == "cuda":
    print(f'- GPU: {torch.cuda.get_device_name(0)} | cuDNN: {torch.backends.
    ↪cudnn.version()}')

print("\nInputs (from 01/02):")
print(f'- Pool rows: {len(source_pool_rel):,}')
print(f'- Class map: {CLASS_TO_INDEX}')
print(f'- Image size: {IMG_SIZE} | Transform mode: {TRANSFORM_MODE}')
print(f'- ImageNet mean/std: {IMAGENET_MEAN} / {IMAGENET_STD}')

print("\nOutputs:")

```

```

print(f'- Checkpoints: {CHECKPOINTS_DIR.relative_to(PROJECT_ROOT)}')
print(f'- Metrics: {RESULTS_DIR.relative_to(PROJECT_ROOT)}')

print("\nReproducibility:")
print(f'- Seed: {SEED} | Deterministic cuDNN: {torch.backends.cudnn.
    ↪deterministic}')
print(f'- Precision: {"Mixed (AMP)" if USE_AMP else "FP32"}')

```

03_model_training - Environment

- Device: CUDA | PyTorch: 2.10.0.dev20250930+cu128 | CUDA: 12.8
- GPU: NVIDIA GeForce RTX 5090 Laptop GPU | cuDNN: 91002

Inputs (from 01/02):

- Pool rows: 13,521
- Class map: {'Pneumonia': 0, 'TB': 1, 'Normal': 2}
- Image size: 256 | Transform mode: letterbox
- ImageNet mean/std: [0.485, 0.456, 0.406] / [0.229, 0.224, 0.225]

Outputs:

- Checkpoints: experiments/checkpoints
- Metrics: experiments/results

Reproducibility:

- Seed: 42 | Deterministic cuDNN: True
- Precision: Mixed (AMP)

0.2 2. Normalisation Toggle.

Select the normalisation source for preprocessing. ##### Rationale: - Enable flexible preprocessing for either pretrained or custom models. - **ImageNet** (**USE_IMAGENET_STATS = True**): reproduces pretrained baselines (MobileNetV2, EfficientNet-B0, ResNet-50). - **CXR** (**USE_IMAGENET_STATS = False**): applies dataset-specific mean and std computed in 01_data_eda.ipynb. - Maintain consistent preprocessing while ensuring fair comparison between pretrained and custom CNN.

```

[14]: import json
      from pathlib import Path

PROJECT_ROOT = Path('.').resolve()
cfg_path = PROJECT_ROOT / "experiments" / "pools" / "preprocessing_config.json"

# Allows flexible selection of mean/std parameters for preprocessing.
# Set USE_IMAGENET_STATS = True to reproduce pretrained baselines (MobileNetV2, U
    ↪EfficientNet-B0, ResNet-50).
# Set to False to use dataset-specific statistics computed in 01_data_eda.ipynb U
    ↪(for custom CNNs).

cfg = json.loads(cfg_path.read_text()) # Choose normalisation source.
USE_IMAGENET_STATS = False # Change manually when needed.

```

```

# Use statistics (brightness and contrast) of the ImageNet dataset, so
↳ pretrained filters see the same data
# distribution they were trained on.
if USE_IMAGENET_STATS:
    MEAN = cfg["imagenet_mean"]
    STD = cfg["imagenet_std"]
    source = "ImageNet"
# Use true brightness/contrast of the medical images from TBX11K and CheXpert
↳ datasets to train models from
# scratch, so normalisation matches Chest X-rays domain.
else:
    MEAN = cfg["cxr_mean"]
    STD = cfg["cxr_std"]
    source = "CXR (dataset-specific)"

print(f"\nUsing {source} normalisation:")
print(f"  mean = {MEAN}")
print(f"  std  = {STD}")

```

Using CXR (dataset-specific) normalisation:

```

mean = [0.5336471796035767, 0.5336471796035767, 0.5336471796035767]
std  = [0.26645559072494507, 0.26645559072494507, 0.26645559072494507]

```

0.3 3. Module Import Path Setup.

Ensure that the project's internal modules stored under '/src' (such as /src/dataset.py, /src/transforms.py, and /src/loaders.py) are discoverable when the notebook is executed from within the /notebooks directory. By explicitly adding the parent directory to sys.path, we make sure that Python recognises the custom package structure regardless of runtime environment (local workstation, remote server, or container). ##### Rationale: - Avoids import errors when running notebooks from nested directories by dynamically adding the project root to the Python path. - Guarantees **modular reproducibility** and **environmental portability** across local and cloud-based executions. - Aligns the notebook workflow with the structure used for later **containerised and Kubernetes deployments**.

```

[15]: import sys
      from pathlib import Path

      # Ensure the project root (one level up from 'notebooks/') is visible to Python
      PROJECT_ROOT = Path("..").resolve()
      if str(PROJECT_ROOT) not in sys.path:
          sys.path.insert(0, str(PROJECT_ROOT))

      print("Project root added to sys.path:", PROJECT_ROOT)

```

Project root added to sys.path: /lab/px/msc-sept-24-ft-cohort-capstone-chrisanich

0.4 4. Shared Preprocessing Transforms.

Construct the shared preprocessing pipelines for both **training** and **evaluation** datasets using the parameters from **preprocessing_config.json** persisted from *01_data_eda.ipynb*. These transforms include deterministic resizing, cropping, and ImageNet-compatible normalisation, ensuring that both **CheXpert** and **TBX11K** images are processed identically. ##### Rationale:

- Enforces **standardised input statistics** across datasets for fair comparison between custom and pretrained models.
- Supports both ImageNet and CXR normalisation modes via the toggle (**USE_IMAGENET_STATS**)
- Promotes **reproducibility** and **consistency** in all subsequent experiments by applying identical preprocessing logic.

```
[16]: # Shared preprocessing transforms (Train / Eval)
# Build shared preprocessing transforms using parameters from
# preprocessing_config.json.
# These transforms ensure consistent resizing, cropping, and ImageNet
# normalisation
# across both datasets (CheXpert and TBX11K).

from src.transforms import build_shared_transforms

train_transforms_shared, eval_transforms_shared = build_shared_transforms(
    img_size=IMG_SIZE,
    mean=MEAN,
    std=STD,
)

print(f'Transforms built successfully: {IMG_SIZE}x{IMG_SIZE}
      ({source}-normalised)')
```

Transforms built successfully: 256x256 (CXR (dataset-specific)-normalised)

0.5 5. Stratified Train/Validation DataLoaders.

Build the final **train** and **validation** DataLoaders by splitting the unified dataset into 80/20 **stratified** partition. This ensures that all three classes, **Pneumonia**, **TB**, and **Normal**, maintain proportional representation in both subsets. Attach the appropriate transform pipeline to each split and configure **DataLoaders** optimised for GPU throughput. ##### Rationale:

- Preserves **class balance** across splits, preventing training bias and enabling robust evaluation.
- Ensures that **data augmentation** is applied only to the **training** subset while **evaluation** remains deterministic.
- Uses **parallel workers** and **pinned memory** to maximise GPU utilisation and minimise I/O latency.
- Facilitates **reproducible, high-throughput training** under fixed seeds and controlled environment conditions.

```
[17]: from src.loaders import build_dataloaders

train_loader, val_loader = build_dataloaders(
```

```

source_pool_rel=source_pool_rel,
class_to_index=CLASS_TO_INDEX,
chexpert_root=CHEXPERT_PATH,
tbx_root=TBX11K_PATH,
img_size=IMG_SIZE,
train_transforms=train_transforms_shared,
eval_transforms=eval_transforms_shared,
seed=SEED,
batch_size=64,
num_workers=4,
)

```

0.6 6. Model Initialisation and Training Configuration.

Define and initialise the neural network architecture, loss function, and optimisation strategy used for model training and evaluation. This section ensures that both **custom** and **pretrained** models follow a unified configuration. ##### Rationale: - Maintain **consistency** across all model types (**custom CNN** and **transfer-learning backbones**). - Enable **reproducible training** by fixing seeds and using identical optimiser and scheduler settings. - Support **fair comparison** of model performance under controlled experimental conditions.

```

[18]: # Model Initialisation and Training Configuration.
# Define the selected model architecture, loss function, optimiser, and
# scheduler.
# This configuration ensures consistent setup for both custom and
# transfer-learning backbones
# (MobileNetV2, Efficient-B0, ResNet-50) as well as the baseline CNN.

from torchvision import models
import torch.nn as nn
import torch.optim as optim

# 1. Selection model architecture.
# Options: 'victorio', 'mobilenet_v2', 'efficientnet_b0', 'resnet50'
MODEL_NAME = "victorio" # Example: can be changed per experiment.

if MODEL_NAME == "mobilenet_v2":
    model = models.mobilenet_v2(weights="IMAGENET1K_V1")
    model.classifier[1] = nn.Linear(model.last_channel, len(CLASS_TO_INDEX))
elif MODEL_NAME == "efficientnet_b0":
    model = models.efficientnet_b0(weights="IMAGENET1K_V1")
    model.classifier[1] = nn.Linear(model.classifier[1].in_features,
# len(CLASS_TO_INDEX))
elif MODEL_NAME == "resnet50":
    model = models.resnet50(weights="IMAGENET1K_V1")
    model.fc = nn.Linear(model.fc.in_features, len(CLASS_TO_INDEX))
elif MODEL_NAME == "victorio":

```

```

# Example baseline CNN (placeholder, will be defined later if needed).
# The name of the custom model is victorio:v1.0 in memory of Dr. Victorio
↳Corradi,
# researcher's medical-doctor mentor in respiratory diseases during his
↳medical training
# at Universidad de Valparaíso and Hospital San Camilo, San Felipe, Chile,
↳2006-2012.
model = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.AdaptiveAvgPool2d((4, 4)), # Force output to 4x4, regardless of
↳input.
                                # AdaptiveAvgPool2d dynamically
↳squeezes each feature map
                                # to 4x4 spatial dimensions. This is
↳what architectures like
                                # ResNet and EfficientNet do. It ensures
↳the fully connected
                                # layer always receives a fixed-length
↳vector.
    nn.Flatten(),
    nn.Linear(32 * 4 * 4, len(CLASS_TO_INDEX)),
)
else:
    raise ValueError(f'Unknown model: {MODEL_NAME}')

# Move model to GPU if available.
model = model.to(device)

```

0.7 7. Debug Confirmation (CUDA and Device Sanity Check).

Verify that the GPU environment is correctly initialised before training. This diagnostic step confirms that PyTorch can access the NVIDIA RTX 5090 device, initialise CUDA contexts, and allocate tensors for computation. # Rationale: - Ensures **CUDA** and **cuDNN** are properly loaded and functional in the active environment. - Confirms **hardware acceleration** is available for model training and evaluation. - Prevents silent fallbacks to CPU execution, maintaining **reproducibility and performance consistency**.

```

[19]: import torch

# GPU diagnostic and model sanity check.
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():

```

```

print("CUDA version:", torch.version.cuda)
print("Device name:", torch.cuda.get_device_name(0))
print("Total GPU memory (GB):", round(torch.cuda.get_device_properties(0).
    ↳total_memory / 1e9, 2))
print("Memory allocated:", torch.cuda.memory_allocated() / 1e9, "GB")
print("Memory reserved:", torch.cuda.memory_reserved() / 1e9, "GB")
else:
    print("CUDA not detected. Training will run on CPU only.")

# Quick sanity check: forward pass of a dummy batch.
x = torch.randn(4, 3, IMG_SIZE, IMG_SIZE).to(device) # Random batch of 4 images.
with torch.no_grad():
    out = model(x)
print("Output shape:", out.shape)

```

```

CUDA available: True
CUDA version: 12.8
Device name: NVIDIA GeForce RTX 5090 Laptop GPU
Total GPU memory (GB): 25.16
Memory allocated: 0.107989504 GB
Memory reserved: 6.905921536 GB
Output shape: torch.Size([4, 3])

```

0.8 8. SWITCH: Scalable Fine-Tuning and Multi-Stage Optimisation Configuration.

Define a dynamic configuration system that enables progressive model development. From short debug tests to pilot and full training stages. Using identical logic and reproducible hyperparameter scaling. This allows fast experimentation and fine-tuning before committing to overnight full-scale runs. *#### Rationale:* - Reduce computational waste by validating configurations in shorter runs. - Enable incremental optimisation of learning rate, batch size, and image resolution. - Maintain reproducibility across scales by controlling all variables from a single cell. - Support fine-tuning and layer unfreezing for transfer-learning architectures.

[20]: *# Provide a progressive system for quick, pilot, and full training stages.*
Ensures identical code behaviour across experiments with scalable parameters.

```

import math
from torch import nn, optim

# Stage selection.
MODE = "pilot" # Options: "quick", "pilot", "full".
FINETUNE = True # Whether to unfreeze pretrained backbone layers.

# Stage-dependent hyperparameters.
if MODE == "quick": # Rapid debugging and stability checks (minutes).
    IMG_SIZE      = 128
    NUM_EPOCHS    = 3 # Number of complete passes through the dataset.

```

```

    BATCH_SIZE    = 32
    BASE_LR       = 3e-4
    WEIGHT_DECAY  = 1e-4
elif MODE == "pilot":
    IMG_SIZE      = 224
    NUM_EPOCHS    = 15
    BATCH_SIZE    = 64
    BASE_LR       = 2e-4
    WEIGHT_DECAY  = 1e-4
else: # Full training (overnight runs).
    IMG_SIZE      = 256
    NUM_EPOCHS    = 60
    BATCH_SIZE    = 128
    BASE_LR       = 1e-4
    WEIGHT_DECAY  = 1e-4

# Fine-tuning behaviour.
if FINETUNE:
    print("Fine-tuning enabled: all backbone layers trainable.")
    for p in model.parameters():
        p.requires_grad = True
else:
    print("Backbone frozen: only classifier layers trainable.")
    if hasattr(model, "features"): # MobileNet / EfficientNet.
        for p in model.features.parameters():
            p.requires_grad = False
    elif hasattr(model, "layer1"): # ResNet family.
        for p in model.layer1.parameters():
            p.requires_grad = False

# Optimiser and scheduler.
optimizer = optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=BASE_LR,
    weight_decay=WEIGHT_DECAY
)

# Cosine-annealing scheduler with linear warm-up.
def build_scheduler(optimizer, num_epochs, warmup_epochs=3):
    def lr_lambda(epoch):
        if epoch < warmup_epochs:
            return (epoch + 1) / warmup_epochs
        progress = (epoch - warmup_epochs) / float(max(1, num_epochs -
↪warmup_epochs))
        return 0.5 * (1.0 + math.cos(math.pi * progress))
    return optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

```

```

scheduler = build_scheduler(optimizer, NUM_EPOCHS)

# Loss function and regularisation tweaks.
criterion = nn.CrossEntropyLoss(label_smoothing=0.05)

for m in model.modules(): # Adjust Dropout / BatchNorm for mode scaling.
    if isinstance(m, nn.Dropout):
        m.p = 0.4 if MODE != "quick" else 0.25
    if isinstance(m, nn.BatchNorm2d):
        m.momentum = 0.1

# Readiness log.
print(f'\nMode: {MODE.upper()} | Fine-tune: {FINETUNE}')
print(f'LR={BASE_LR:.1e} | WD={WEIGHT_DECAY:.1e} | Epochs={NUM_EPOCHS} |   

↪Batch={BATCH_SIZE}')
print(f'Image size={IMG_SIZE}px | Precision={"AMP" if USE_AMP else "FP32"}')
print("Optimiser and scheduler initialised: ready for Training Loop.")

```

Fine-tuning enabled: all backbone layers trainable.

```

Mode: PILOT | Fine-tune: True
LR=2.0e-04 | WD=1.0e-04 | Epochs=15 | Batch=64
Image size=224px | Precision=AMP
Optimiser and scheduler initialised: ready for Training Loop.

```

0.9 9. Pre-Training Diagnostics and Dry-Run Validation.

Before executing the full training routine, a short diagnostic stage ensures that all components, datasets, precision settings, and checkpoint directories, are properly configured. Running these quick checks prevents wasting hours of computation due to silent configuration or runtime errors.
Rationale: - Confirm **DataLoaders** yielded batches of the expected size and format. - Validate **model forward/backward pass** (detecting shape or precision mismatches). - Ensure **checkpoint and results directories** exist before training. - Execute a **one-batch dry run** to measure memory usage and prevent out-of-memory crashes.

```

[21]: # Pre-Training Diagnostics and Dry-Run Validation.
# Ensures data, model, and optimisation pipeline are operational
# before launching long training runs (detects precision or device issues   

↪early).

import torch, os, time
from torch.amp import autocast, GradScaler

# 1. Directory readiness.
for folder in [CHECKPOINTS_DIR, RESULTS_DIR]:
    folder.mkdir(parents=True, exist_ok=True)
    print(f'- Folder ready: {folder}')

```

```

# 2. DataLoader sample inspection.
# Retrieve one mini-batch to confirm tensor shapes and device transfer.
images, labels = next(iter(train_loader))
print(f'Sample batch shape: {tuple(images.shape)} | Labels: {labels[:5]}.
      ↪tolist()}')

# 3. Model forward / backward dry-run.
USE_AMP = True
scaler = GradScaler("cuda", enabled=USE_AMP)

images, labels = images.to(device), labels.to(device)

# autocast uses context syntax: autocast("cuda", enabled=USE_AMP)
with autocast("cuda", enabled=USE_AMP):
    outputs = model(images)
    loss = criterion(outputs, labels)

print(f'Forward pass OK. Loss: {loss.item():.4f}')

# Gradient flow validation.
optimizer.zero_grad(set_to_none=True)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()

print("Backward + optimiser step OK. Gradients flowing correctly.")

# 4. Optional one-epoch smoke test.
# (Uncomment for a fast 1-epoch trial to confirm end-to-end flow).
# NUM_EPOCHS = 1
# _ = run_epoch(train_loader, model, criterion, optimizer, train=True)

print("\nPre-training diagnostics completed successfully. System ready for full
      ↪training run.")

```

- Folder ready: /lab/px/msc-sept-24-ft-cohort-capstone-chrisanich/experiments/checkpoints

- Folder ready: /lab/px/msc-sept-24-ft-cohort-capstone-chrisanich/experiments/results

Sample batch shape: (64, 3, 256, 256) | Labels: [1, 1, 0, 2, 2]

Forward pass OK. Loss: 1.1004

Backward + optimiser step OK. Gradients flowing correctly.

Pre-training diagnostics completed successfully. System ready for full training run.

0.10 10. Perceptual Image-Hash Test for Data Leakage Detection

This diagnostic verifies that the training and validation splits do not share any duplicate images, even if filenames differ. Instead of comparing filenames, the method computes a SHA-256 hash of a downsampled grayscale version of each image, ensuring that visually identical images are detected regardless of directory structure or naming. ##### Rationale: - Ensures there is no accidental overlap between training and validation data. - Detects duplicates by image content, not file name or path. - Prevents artificially inflated accuracy caused by repeated images across splits. - Guarantees that evaluation metrics reflect true generalisation, not leakage.

```
[22]: # 10. Perceptual Image-Hash Test for Data Leakage Detection.

import os, hashlib
from PIL import Image

def hash_image(path):
    """Return a deterministic SHA256 hash after grayscale + downsample."""
    img = Image.open(path).convert("L")
    img = img.resize((64, 64))
    return hashlib.sha256(img.tobytes()).hexdigest()

def collect_hashes(dataset):
    """
    Generate perceptual hashes for all images in a dataset split.

    The CXRThreeClassDataset stores:
        dataset.items["rel_path"]
        dataset.items["Source"]
    Absolute paths must be reconstructed exactly as in __getitem__.
    """
    hashes = []
    for i in range(len(dataset)):
        row = dataset.items.iloc[i]

        if row["Source"] == "chexpert":
            abs_path = os.path.join(dataset.chexpert_root, row["rel_path"])
        else:
            abs_path = os.path.join(dataset.tbx_root, "imgs", row["rel_path"])

        hashes.append((hash_image(abs_path), row["Class"], row["rel_path"]))
    return hashes

print("Hashing TRAIN...")
train_hashes = collect_hashes(train_loader.dataset)

print("Hashing VAL...")
val_hashes = collect_hashes(val_loader.dataset)
```

```

train_hash_set = {h[0] for h in train_hashes}
val_hash_set    = {h[0] for h in val_hashes}

overlap_train_val = train_hash_set & val_hash_set

print("\n--- IMAGE-HASH DUPLICATE LEAKAGE REPORT ---")
print(f"Train vs Val duplicates: {len(overlap_train_val)}")

if len(overlap_train_val) == 0:
    print("\nNo duplicates found: Dataset split is clean.")
else:
    print("\nWarning: duplicates detected between train and validation.")

```

Hashing TRAIN...

Hashing VAL...

--- IMAGE-HASH DUPLICATE LEAKAGE REPORT ---

Train vs Val duplicates: 0

No duplicates found - dataset split is clean.

0.11 11. Training Loop and Metrics Tracking.

Train and validate the selected model over multiple epochs while monitoring loss and accuracy. This section implements a reproducible supervised training loop compatible custom and transfer-learning architectures. ##### Rationale: - Executes **iterative optimisation** of network weights over mini-batches. - Monitors **training vs validation performance** to detect overfitting. - Saves best checkpoints and metrics for later **statistical testing** and reproducibility.

[24]: # 1. Configuration parameters.

```

from torch.amp import autocast, GradScaler
from tqdm import tqdm
import time, json

# 1. Configuration parameters.
best_val_acc = 0.0 # Track best validation accuracy achieved so far.
# USE: Automatic Mixed Precision package - torch.amp
# Some ops, like linear layers and convolutions, are much faster in
    ↳ 'lower_precision_fp'. Other ops, like reductions,
# often require the dynamic range of 'float32'. Mixed precision tries to match
    ↳ each op to its appropriate datatype.
scaler = GradScaler(device="cuda", enabled=USE_AMP) # Handle safe scaling when
    ↳ mixed automatic main precision (AMP) is enabled.

# Initialise a metrics dictionary to record training dynamics across epochs.
history = {

```

```

    "epoch": [],
    "train_loss": [],
    "val_loss": [],
    "train_acc": [], # Training accuracy.
    "val_acc": [], # Validation accuracy.
}

# 2. Helper functions.
def accuracy(preds, labels): # preds(logits): raw output numbers that a
    ↪ machine learning models produces before
                                # converting them into probabilities. They are
    ↪ like confidence scores, not yet
                                # squished into a range between 0 and 1.
    """
    Compute classification accuracy as the mean proportion of correct
    ↪ predictions.
    Args:
        preds (Tensor): Model output logits for each class.
        labels (Tensor): Ground-truth class indices.
    Returns:
        float: Accuracy in range [0, 1].
    """
    return (preds.argmax(dim=1) == labels).float().mean().item()

def run_epoch(loader, model, criterion, optimizer=None, train=False):
    """
    Execute one full pass (epoch) over a DataLoader, performing training or
    ↪ evaluation.

    Args:
        loader (DataLoader): The dataset iterator (train or validation).
        model (nn.Module): Current neural network instance.
        criterion: Loss function used to quantify prediction error.
        optimizer (optional): Optimiser for gradient descent (only if training).
                                Gradient is the vector of partial derivatives of
    ↪ the
                                loss with respect to each parameter. It
    ↪ essentially
                                tells which direction is downhill in the slope of
    ↪ the
                                loss function in the higher dimensional space of
    ↪ the
                                networks's weights.
                                By computing the gradient of the loss it is
    ↪ possible

```

to adjust each weight in order to reduce the
 error a little bit.
 Back propagation is the algorithm that efficiently computes these gradients for all the weights in a multilayer neural network.
 After the network does a forward pass on some training data, and the loss function produces a loss value or the error, back propagation works backwards from the output layer to the input. This is essentially the chain rule of calculus

applied to the network's composed functions breaking down
 the losses derivative layer by layer.
 The result of back propagation is the gradient of
 the loss with respect to every weight and bias in the
 network.

calculating how much each weight contributed to the error.
 train (bool): true enables gradient updates: False disables them.

Returns:
 tuple: (average_loss, average_accuracy)

```

"""
model.train(train) # Switch model to training or evaluation mode.
total_loss, total_acc = 0.0, 0.0

# tqdm provides a real-time progress bar for clarity during long runs.
# Tqdm is a Python library that provides fast, extensible progress bars for
loops and
# iterables. It is a simple way to track the advancement of time-intensive
# tasks. The library's name means "progress" in Arabic, and is an
abbreviation for
# "I love you so much" in Spanish (te quiero demasiado).
for images, labels in tqdm(loader, desc=f"{'Train' if train else 'Val'}",
leave=False):
    # Move data to GPU for acceleration (if available).
    images, labels = images.to(device), labels.to(device)

    # Automatic Mixed Precision (AMP): uses lower precision during forward
pass
    # to improve speed and reduce memory use, while maintaining numeric
stability.

```

```

    # Automatic Mixed Precision (AMP) is a technique for deep learning that
    ↳ combines the speed
    # of half-precision (FP16) floating-point numbers with the accuracy of
    ↳ single-precision (FP32)
    # to accelerate model training and reduce memory usage with minimal
    ↳ programming effort.
    # AMP frameworks like PyTorch and TensorFlow automatically handle the
    ↳ conversion of operations
    # to different precisions and implement techniques like loss scaling to
    ↳ prevent gradient underflow,
    # ensuring accuracy is maintained. This allows users to leverage
    ↳ hardware features, such as
    # NVIDIA Tensor Cores, for faster times.
    with autocast(device_type="cuda", enabled=USE_AMP):
        outputs = model(images) # Forward propagation (prediction step).
        loss = criterion(outputs, labels) # Compute loss against ground
        ↳ truth.

    if train:
        # Reset gradients from previous batch before backpropagation.
        optimizer.zero_grad()

        # Scale loss to prevent underflow in float16 computations (AMP
        ↳ only).
        scaler.scale(loss).backward() # Compute gradients via
        ↳ backpropagation.

        # Apply the scaled gradients to update model parameters.
        scaler.step(optimizer)

        # Update the gradient scaler dynamically (helps AMP stability).
        scaler.update()

        # Accumulate mean metrics over all batches for epoch summary.
        total_loss += loss.item()
        total_acc += accuracy(outputs, labels)

    # Compute epoch-averaged metrics (loss and accuracy).
    return total_loss / len(loader), total_acc / len(loader)

# 3. Training loop.
print(f'\nStarting training for {NUM_EPOCHS} epochs on {device} ...')
start_time = time.time() # Record start time to compute total training duration.

# --- Early stopping parameters ---
# This is a training control technique that stops training automatically when

```

```

# validation loss stops improving.
# Early stopping prevents overfitting from extra epochs that only memorise
↳ training set.
# This could save hours and keep the best-performing model automatically.
# Early stopping saves time if val loss plateaus early. It prevents overfitting
↳ and wasted
# GPU time.
patience = 8          # Stop after 8 epochs without improvements.
epochs_no_improve = 0 # Counter for consecutive non-improvements.

for epoch in range(1, NUM_EPOCHS + 1):
    # Training phase.
    train_loss, train_acc = run_epoch(
        train_loader, model, criterion, optimizer, train=True
    )

    # Validation phase.
    val_loss, val_acc = run_epoch(
        val_loader, model, criterion, train=False
    )

    # Step the learning rate scheduler (e.g., decay every few epochs).
    scheduler.step()

    # Log metrics to history for later analysis and visualisations.
    history["epoch"].append(epoch)
    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)

    # Print concise summary for this epoch.
    print(
        f'Epoch: {epoch:02d}/{NUM_EPOCHS} | '
        f'Train Loss: {train_loss:.4f}, Accuracy: {train_acc:.4f} | '
        f'Val Loss: {val_loss:.4f}, Accuracy: {val_acc:.4f}'
    )

    # Checkpoint and early stopping logic.
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        epochs_no_improve = 0 # Reset counter.
        # --- CHECKPOINT SAVING (mode-dependent naming).
        domain = "imagenet" if USE_IMAGENET_STATS else "cxr"
        if MODE == "full":
            # Full training run → canonical best model filename
            ckpt_path = CHECKPOINTS_DIR / f'{MODEL_NAME}_{domain}_best.pt'

```

```

else:
    # Quick / Pilot runs → keep mode in filename
    ckpt_path = CHECKPOINTS_DIR / f'{MODEL_NAME}_{domain}_{MODE}.pt'

    torch.save(model.state_dict(), ckpt_path)
    print(f'--- New best model saved --> {ckpt_path.name} (val_acc={val_acc:
↪.4f})')

else:
    epochs_no_improve += 1
    print(f'No improvement for {epochs_no_improve}/{patience} epochs.')

    # Trigger early stopping if no improvement for "patience" epochs.
    if epochs_no_improve >= patience:
        print(f'\nEarly stopping triggered at epoch {epoch}.')
        break
    # -----

# 4. Save metrics history.
# Persist the training history as JSON for later statistical comparison and
↪plotting.
# Save metrics with matching naming rule.
domain = "imagenet" if USE_IMAGENET_STATS else "cxr"

if MODE == "full":
    results_path = RESULTS_DIR / f'{MODEL_NAME}_{domain}_best_metrics.json'
else:
    results_path = RESULTS_DIR / f'{MODEL_NAME}_{domain}_{MODE}_metrics.json'

with open(results_path, "w") as f:
    json.dump(history, f, indent=2)

# Print final summary of training session.
elapsed = (time.time() - start_time) / 60
print(f'\nTraining completed in {elapsed:.2f} minutes.')
print(f'Best validation accuracy: {best_val_acc:.4f}')
print(f'Metrics saved to: {results_path.relative_to(PROJECT_ROOT)}')

```

Starting training for 15 epochs on cuda ...

Epoch: 01/15 | Train Loss: 0.4283, Accuracy: 0.8957 | Val Loss: 0.4343,
Accuracy: 0.8938
--- New best model saved --> victorio_cxr_pilot.pt (val_acc=0.8938)

Epoch: 02/15 | Train Loss: 0.4289, Accuracy: 0.8947 | Val Loss: 0.4346,
Accuracy: 0.8960
--- New best model saved --> victorio_cxr_pilot.pt (val_acc=0.8960)

Epoch: 03/15 | Train Loss: 0.4285, Accuracy: 0.8947 | Val Loss: 0.4336,
Accuracy: 0.8949
No improvement for 1/8 epochs.

Epoch: 04/15 | Train Loss: 0.4281, Accuracy: 0.8951 | Val Loss: 0.4331,
Accuracy: 0.8934
No improvement for 2/8 epochs.

Epoch: 05/15 | Train Loss: 0.4268, Accuracy: 0.8956 | Val Loss: 0.4299,
Accuracy: 0.8952
No improvement for 3/8 epochs.

Epoch: 06/15 | Train Loss: 0.4249, Accuracy: 0.8958 | Val Loss: 0.4280,
Accuracy: 0.8960
No improvement for 4/8 epochs.

Epoch: 07/15 | Train Loss: 0.4206, Accuracy: 0.8977 | Val Loss: 0.4252,
Accuracy: 0.8963
--- New best model saved --> victorio_cxr_pilot.pt (val_acc=0.8963)

Epoch: 08/15 | Train Loss: 0.4173, Accuracy: 0.9007 | Val Loss: 0.4203,
Accuracy: 0.8981
--- New best model saved --> victorio_cxr_pilot.pt (val_acc=0.8981)

Epoch: 09/15 | Train Loss: 0.4130, Accuracy: 0.9019 | Val Loss: 0.4202,
Accuracy: 0.9000
--- New best model saved --> victorio_cxr_pilot.pt (val_acc=0.9000)

Epoch: 10/15 | Train Loss: 0.4081, Accuracy: 0.9027 | Val Loss: 0.4113,
Accuracy: 0.9000
No improvement for 1/8 epochs.

Epoch: 11/15 | Train Loss: 0.4058, Accuracy: 0.9038 | Val Loss: 0.4071,
Accuracy: 0.9057
--- New best model saved --> victorio_cxr_pilot.pt (val_acc=0.9057)

```
Epoch: 12/15 | Train Loss: 0.4009, Accuracy: 0.9079 | Val Loss: 0.4037,
Accuracy: 0.9065
--- New best model saved --> victorio_cxr_pilot.pt (val_acc=0.9065)
```

```
Epoch: 13/15 | Train Loss: 0.4004, Accuracy: 0.9061 | Val Loss: 0.3973,
Accuracy: 0.9080
--- New best model saved --> victorio_cxr_pilot.pt (val_acc=0.9080)
```

```
Epoch: 14/15 | Train Loss: 0.3954, Accuracy: 0.9106 | Val Loss: 0.4018,
Accuracy: 0.9054
No improvement for 1/8 epochs.
```

```
Epoch: 15/15 | Train Loss: 0.3919, Accuracy: 0.9102 | Val Loss: 0.3937,
Accuracy: 0.9094
--- New best model saved --> victorio_cxr_pilot.pt (val_acc=0.9094)
```

```
Training completed in 6.11 minutes.
Best validation accuracy: 0.9094
Metrics saved to: experiments/results/victorio_cxr_pilot_metrics.json
```

0.12 12. Training Loop and Metrics Tracking.

Train and validate the selected model over multiple epochs while monitoring loss and accuracy. This section implements a reproducible supervised training loop compatible with both custom and transfer-learning architectures. ##### Rationale: - Executes iterative optimisation of network weights over mini-batches using backpropagation and gradient-based updates. - Monitors training and validation loss/accuracy at each epoch to detect underfitting, overfitting, or training instabilities. - Applies scheduled learning-rate updates and early stopping to improve convergence and prevent unnecessary computation. - Saves the best-performing model checkpoint based on validation accuracy for later evaluation, explainability and deployment. - Logs per-epoch metrics to JSON files, enabling statistical comparison across models, modes (quick/pilot/full), and normalisation domains (ImageNet vs CXR).

```
[12]: # Quick sanity check for data leakage + generalisation.

from torch.utils.data import Subset

# 1. Check if train and val share any underlying indices (leakage test).
train_ds = train_loader.dataset
val_ds   = val_loader.dataset

overlap_count = None
```

```

try:
    train_idx = set(train_ds.indices) if isinstance(train_ds, Subset) else None
    val_idx = set(val_ds.indices) if isinstance(val_ds, Subset) else None
    if train_idx is not None and val_idx is not None:
        overlap_count = len(train_idx & val_idx)
except AttributeError:
    pass

print(f"Train dataset type: {type(train_ds)}")
print(f"Val dataset type: {type(val_ds)}")
if overlap_count is not None:
    print(f"Number of overlapping samples between train and val:␣
↪{overlap_count}")
    if overlap_count > 0:
        print("WARNING: train/val leakage detected.")
else:
    print("Could not inspect indices (dataset is not a torch.utils.data.Subset).
↪")

# 2. Evaluate accuracy on TRAIN and VAL with the current model (no gradient).
model.eval()
with torch.no_grad():
    train_loss_check, train_acc_check = run_epoch(train_loader, model,␣
↪criterion, train=False)
    val_loss_check, val_acc_check = run_epoch(val_loader, model,␣
↪criterion, train=False)

print(f"\nSanity evaluation:")
print(f"Train loss: {train_loss_check:.4f} | Train acc: {train_acc_check:.4f}")
print(f"Val loss: {val_loss_check:.4f} | Val acc: {val_acc_check:.4f}")

```

```

Train dataset type: <class 'src.dataset.CXRThreeClassDataset'>
Val dataset type: <class 'src.dataset.CXRThreeClassDataset'>
Could not inspect indices (dataset is not a torch.utils.data.Subset).

```

```

Sanity evaluation:
Train loss: 0.1700 | Train acc: 1.0000
Val loss: 0.1861 | Val acc: 0.9931

```

0.13 13. Metric Visualisation and Statistical Summary.

Visualise and compare the performance of all trained models (both custom and pretrained) by automatically loading every `*_metrics.json` file stored in the ‘experiments/results directory. Each metrics file contains the recorded training and validation losses and accuracies per epoch. #####

Rationale: - Enables direct performance comparison between models and normalisation modes (ImageNet vs CXR). - Evaluates convergence behaviour and potential overfitting trends. - Provides a compact statistical overview of validation accuracy and loss for all architectures. - Facilitates reproducible benchmarking across experiments.

```
[ ]: # Cross Model Comparison (Ordered by Best Accuracy, Boxed Legend)
# This script compares all model runs by plotting training and validation
# accuracy per epoch.
# It follows Tufte's principles of clarity and minimalism:
# 1. Show only necessary data (accuracy curves).
# 2. Use consistent axes and spacing for visual comparison.
# 3. Order models by best validation accuracy (descending).
# 4. Keep text minimal but precise, with a simple boxed legend.

import json, csv
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

# Define paths for project folders and ensure they exist.
PROJECT_ROOT = Path("..").resolve()
RESULTS_DIR = PROJECT_ROOT / "experiments" / "results"
FIG_DIR = PROJECT_ROOT / "reports" / "figures"
TABLE_DIR = PROJECT_ROOT / "reports" / "tables"
for d in [FIG_DIR, TABLE_DIR]:
    d.mkdir(parents=True, exist_ok=True)

# Load all training metrics JSON files.
files = list(RESULTS_DIR.glob("*_metrics.json"))
if not files:
    raise FileNotFoundError("No metrics files found in the results directory")

results = {}
for path in files:
    name = path.stem.replace("_metrics", "")
    with open(path) as f:
        h = json.load(f)
    e = np.array(h["epoch"])
    train, val = np.array(h["train_acc"]), np.array(h["val_acc"])
    results[name] = {
        "epochs": e,
        "train": train,
        "val": val,
        "best_val": float(np.max(val)),
        "epoch_best": int(np.argmax(val) + 1),
        "final_val": float(val[-1]),
    }
```

```

# Order models by best validation accuracy (descending).
sorted_models = sorted(results.keys(), key=lambda x: results[x]["best_val"],
    ↪reverse=True)

# Plot accuracy curves for training and validation.
fig, axes = plt.subplots(1, 2, figsize=(10, 5), sharey=True)
ax_train, ax_val = axes

for name in sorted_models:
    e = results[name]["epochs"]
    ax_train.plot(e, results[name]["train"], lw=1.8, label=name)
    ax_val.plot(e, results[name]["val"], lw=1.8, label=name)

# Configure both subplots with minimal styling.
for ax, title in zip([ax_train, ax_val], ["Training Accuracy", "Validation_
    ↪Accuracy"]):
    ax.set_title(title)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Accuracy")
    for side in ("top", "right"):
        ax.spines[side].set_visible(False)
    ax.grid(True, alpha=0.2)
    ax.legend(fontsize=8, loc="lower right", frameon=True, framealpha=0.9,
    ↪edgecolor="black")

plt.tight_layout()
fig_path = FIG_DIR / "cross_model_comparison_train_val.png"
plt.savefig(fig_path, dpi=300, bbox_inches="tight")
plt.show()
print(f'Figure saved to: {fig_path.relative_to(PROJECT_ROOT)}')

# Print ordered summary table for quick inspection.
print("\nSummary of Validation Accuracy (Ordered by Best Accuracy):\n")
print(f"{'Model':30s} | {'Best Acc':>8s} | {'Epoch':>5s} | {'Final Acc':>10s}")
print("-" * 60)
for name in sorted_models:
    r = results[name]
    print(f"{'name':30s} | {r['best_val']:.4f} | {r['epoch_best']:5d} |
    ↪{r['final_val']:.4f}")

# Export same summary to CSV for report inclusion.
csv_path = TABLE_DIR / "cross_model_summary.csv"
with open(csv_path, "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["Model", "Best_Val_Acc", "Epoch_Best", "Final_Val_Acc"])

```

```
for name in sorted_models:
    r = results[name]
    w.writerow([name, r["best_val"], r["epoch_best"], r["final_val"]])
print(f'\nTable saved to: {csv_path.relative_to(PROJECT_ROOT)}')
```

```
[ ]:
```